



Milan Jovanović



RACE CONDITIONS

EF CORE OPTIMISTIC CONCURRENCY



Resolvendo condições de corrida com o bloqueio otimista do EF Core

5 minutos de
leitura

9 DE SETEMBRO DE
2023



Gerenciar histórico de
leitura

O EF Core está muito lento? Descubra como inserir dados até 14 vezes mais rápido (reduzindo o tempo de carregamento em 94%). Aumente o desempenho com nosso método integrado ao EF Core: Inserção, atualização, exclusão e mesclagem em massa. Junte-se aos mais de 5.000 clientes satisfeitos que confiam em nossa biblioteca desde 2014. [Clique aqui para saber mais](#).

**Milan Jovanović**

Você desenvolve sistemas de software complexos? Veja como **o NServiceBus** facilita o projeto, a construção e o gerenciamento de sistemas de software que utilizam filas de mensagens para alcançar baixo acoplamento. **Comece GRATUITAMENTE.**

Apoie esta newsletter →

Com que frequência você pensa em conflitos de concorrência ao escrever código?

Você escreve o código para uma nova funcionalidade, confirma que ela funciona e encerra o trabalho por hoje.

Mas uma semana depois, você descobre que introduziu um bug grave porque não pensou na concorrência.

O problema mais comum são as condições de corrida, com duas threads concorrentes executando a mesma função. Se você não levar isso em consideração durante o desenvolvimento, corre o risco de deixar o sistema em um estado corrompido.

No boletim informativo desta semana, desafio você a identificar a condição de corrida em um método de reserva. O requisito comercial é que não pode haver duas reservas sobrepostas para as mesmas datas.

Em seguida, mostrarei como resolver essa condição de corrida usando a concorrência otimista do EF Core.

Vamos começar!

**Milan Jovanović**

Existe uma condição de corrida escondida em algum lugar neste trecho de código.

Você consegue ver?

```
public Result<Guid> Handle(
    ReserveBooking command,
    AppDbContext dbContext)
{
    var user = dbContext.Users.GetById(command.UserId);
    var apartment = dbContextApartments.GetById(command.ApartmentId);
    var (startDate, endDate) = command;

    if (dbContext.Bookings.IsOverlapping(apartment, startDate, endDate))
    {
        return Result.Failure<Guid>(BookingErrors.Overlap);
    }

    var booking = Booking.Reserve(apartment, user, startDate, endDate);

    dbContext.Add(booking);

    dbContext.SaveChanges();

    return booking.Id;
}
```

A chamada **IsOverlapping** é uma verificação otimista para ver se existe uma reserva para as datas especificadas.

```
if (dbContext.Bookings.IsOverlapping(apartment, startDate, endDate)) { }
```

Se retornar `false` **true**, estamos tentando reservar o apartamento duas vezes. Portanto, retornamos uma falha e o método é concluído.

Mas se retornar **false**, reservamos um horário e fazemos uma chamada **SaveChanges** para salvar as alterações no banco de dados.



`IsOverlapping` verificação e tentar reservar a reserva. Sem nenhum controle de concorrência, ambas as solicitações serão bem-sucedidas e acabaremos com um estado inconsistente no banco de dados.

Então, como podemos resolver isso?

Concorrência otimista com EF Core

A abordagem de concorrência pessimista adquire um bloqueio para os dados antes de modificá-los. É mais lenta e faz com que transações concorrentes sejam bloqueadas até que o bloqueio seja liberado. O EF Core não oferece suporte a essa abordagem nativamente.

Você também pode resolver esse problema usando concorrência otimista com o EF Core. Ele não requer nenhum bloqueio, mas qualquer modificação nos dados falhará ao ser salva se os dados tiverem sido alterados desde a última consulta.

Para implementar a concorrência otimista no EF Core, você precisa configurar uma propriedade como um *token de concorrência*. Ela é carregada e rastreada com a entidade. Quando você chama o método `getConcurrencyToken()`

`SaveChanges`, o EF Core compara o valor do token de concorrência com o valor no banco de dados.

Vamos supor que estamos usando o SQL Server, que possui uma `rowversion` coluna nativa. O valor dessa `rowversion` coluna muda automaticamente quando a linha é atualizada, sendo, portanto, uma ótima opção para um token de concorrência.

Para configurar uma `byte[]` propriedade como um token de concorrência, você pode decorá-la com o `Timestamp` atributo. Ela será mapeada para uma

**Milan Jovanović**

```
public class Apartment
{
    public Guid Id { get; set; }

    [Timestamp]
    public byte[] Version { get; set; }
}
```

Prefiro uma abordagem diferente porque os atributos poluem a entidade.

Você pode fazer o mesmo com a Fluent API. Eu até usarei uma propriedade sombra para ocultar o token de concorrência da classe de entidade.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Apartment>()
        .Property<byte[]>("Version")
        .IsRowVersion();
}
```

A configuração exata varia de acordo com o banco de dados que você está usando, portanto, consulte a documentação.

Como a concorrência otimista funciona na prática

Então, veja o que muda quando configuramos o token de simultaneidade.

Ao carregar a **Apartment** entidade, o EF também carregará o token de simultaneidade.

```
SELECT a.Id, a.Version
FROM Apartments a
WHERE a.Id = @p0
```



dados.

```
UPDATE Apartments a
SET a.LastBookedOnUtc = @p0
WHERE a.Id = @p1 AND a.Version = @p2;
```

Se o valor `rowversion` no banco de dados for alterado, o número de linhas atualizadas será `0`.

O EF Core espera atualizar `1` a linha, então ele lançará uma exceção `DbUpdateConcurrencyException`, que você precisa tratar.

Tratamento de exceções de concorrência

Agora que você sabe como usar a concorrência otimista com o EF Core, pode corrigir o trecho de código anterior.

Se duas solicitações simultâneas passarem na `IsOverlapping` verificação, apenas uma poderá concluir a `SaveChanges` chamada. A outra solicitação simultânea encontrará uma `Version` incompatibilidade no banco de dados e lançará uma exceção `DbUpdateConcurrencyException`.

Em caso de conflito de concorrência, precisamos adicionar uma `try-catch` instrução para capturar a exceção `DbUpdateConcurrencyException`. A forma como você lida com a exceção em si depende dos requisitos do seu negócio. E, às vezes, `condições de corrida` podem nem existir.

```
public Result<Guid> Handle(
    ReserveBooking command,
    AppDbContext dbContext)
{
    var user = dbContext.Users.GetById(command.UserId);
    var apartment = dbContextApartments.GetById(command.ApartmentId);
```



```
        return Result.Failure<Guid>(BookingErrors.Overlap);
    }

    try
    {
        var booking = Booking.Reserve(apartment, user, startDate, endDate);

        dbContext.Add(booking);

        dbContext.SaveChanges();

        return booking.Id;
    }
    catch (DbUpdateConcurrencyException)
    {
        return Result.Failure<Guid>(BookingErrors.Overlap);
    }
}
```

Se você está se perguntando como isso vai funcionar, aqui está a peça que faltava. O `Booking.Reserve` método atualizará a `LastBookedOnUtc` propriedade da `Apartment` entidade.

```
public static Booking Reserve(Apartment apartment, User user, DateTime startDate, DateTime endDate)
{
    apartment.LastBookedOnUtc = DateTime.UtcNow;

    return new Booking
    {
        Id = Guid.NewGuid(),
        Apartment = apartment,
        User = user,
        StartDate = startDate,
        EndDate = endDate
    };
}
```

Ao chamarmos `SaveChanges`, isso causará uma atualização na `Apartment` entidade, que incluirá a verificação do token de concorrência. Isso garante que, se duas solicitações tentarem atualizar a mesma entidade



Quando você deve usar a concorrência otimista?

A concorrência otimista considera que o melhor cenário também é o mais provável. Ela assume que os conflitos entre transações serão infrequentes e não adquire bloqueios nos dados. Isso significa que seu sistema pode escalar melhor, pois não há bloqueios que prejudiquem o desempenho.

No entanto, você ainda deve esperar conflitos de concorrência e implementar lógica personalizada para lidar com eles.

A concorrência otimista é uma boa escolha se sua aplicação não prevê muitos conflitos.

Outro motivo para usar a concorrência otimista é quando você não pode manter uma conexão aberta com o banco de dados durante toda a duração da transação. Isso é necessário para o bloqueio pessimista.

Espero que isso tenha sido útil.

Te vejo na próxima semana!

Comentários

Faça login para deixar um comentário



Iniciar sessão com o Google



Faça login com o GitHub



Quando você estiver pronto, existem 4 maneiras pelas quais posso te ajudar:

1. **Arquitetura Limpa Pragmática:** Junte-se a mais de 4.900 alunos neste curso abrangente que ensinará o sistema que utilizo para lançar aplicações prontas para produção usando Arquitetura Limpa. Aprenda a aplicar as melhores práticas da arquitetura de software moderna.
2. **Arquitetura Monolítica Modular:** Junte-se a mais de 2.800 engenheiros neste curso aprofundado que transformará a maneira como você constrói sistemas modernos. Você aprenderá as melhores práticas para aplicar a arquitetura monolítica modular em um cenário real.
3. **APIs REST pragmáticas:** Junte-se a mais de 1.800 alunos neste curso que ensinará como criar APIs REST prontas para produção usando os recursos mais recentes do ASP.NET Core e as melhores práticas. Ele inclui um aplicativo de interface do usuário totalmente funcional que integraremos à API REST.
4. **Comunidade Patreon:** Junte-se a uma comunidade com mais de 5.000 engenheiros e arquitetos de software. Você também terá acesso ao código-fonte que uso nos meus vídeos do YouTube, acesso antecipado a vídeos futuros e descontos exclusivos nos meus cursos.

**Milan Jovanović**

Junte-se a mais de 73.000 engenheiros que estão aprimorando suas habilidades todos os sábados de manhã.

[Join 70K+ Engineers](#)

© 2026 Milan Jovanovic Tech DOO

Contato

